

Relatório de Verificação

Análise Estática de Código

Nome: João Pedro Carvalho dos Santos

Turma: Desenvolvimento de Software Multiplataforma - 6º semestre Data: 12/09/2026

Projeto 1 - Minicurso Flask - <https://github.com/pallets/flask>

1. Detalhamento dos Passos de Execução da Análise

1. Clonei o repositório oficial `pallets/flask` e confirmei que ele atende ao porte exigido pela atividade. Em seguida, criei o projeto `br.edu.fatec:minicurso-flask` no SonarQube, configurei o arquivo `sonar-project.properties` e executei o scanner Python com o token fornecido por variável de ambiente. Após registrar a análise inicial, corriji três issues pontuais, rodei 72 testes direcionados e enviei a versão 1.1 para uma segunda análise.

2. Visão Sumarizada (Overall)

1. Análise inicial (v0), realizada antes das correções. O painel apresentou os seguintes resultados:

Métrica	Resultado inicial
Quality Gate	Passed / OK
Security	3 issues — D
Reliability	18 issues — E
Maintainability	55 issues — A
Accepted Issues	0
Coverage	0,0% — 3.246 linhas a cobrir
Duplications	0,3% — 28 linhas duplicadas
Security Hotspots	0 — A
Linhas de código	5.296 ncloc
Violações totais	63

- 1. Evidência da visão Overall inicial: painel do SonarQube mostrado na Figura 1.

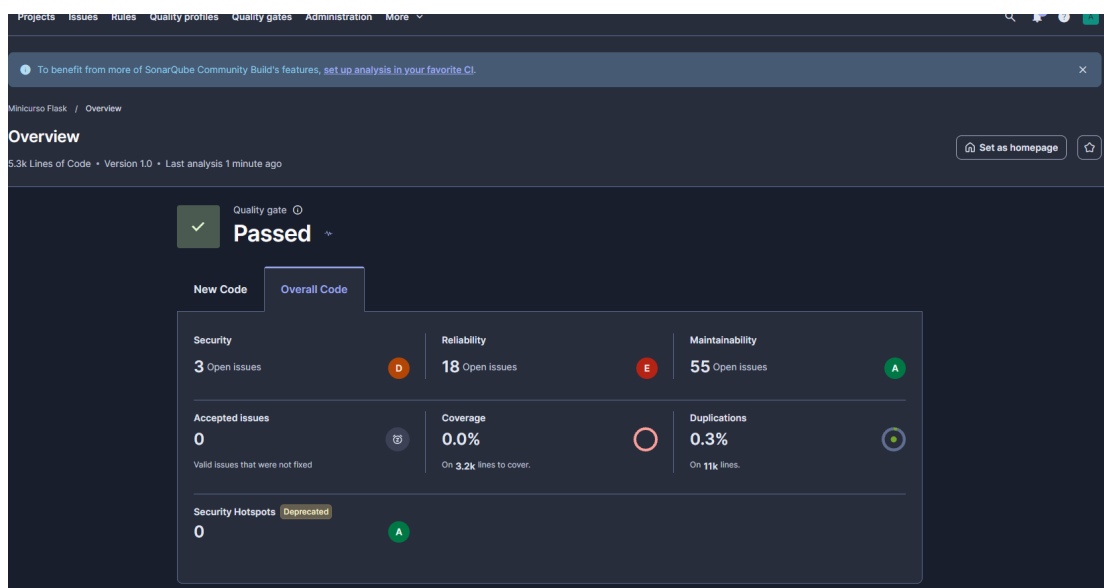


Figura 1 — Visão Overall inicial, antes das correções.

- 1. Quantidade total de linhas de código: 5.296 ncloc, aproximadamente 5,3 mil linhas no painel.

1. Visão Sumarizada (New Code Lines)

- Foram feitas três alterações pontuais: remoção de um yield inútil em tests/conftest.py; atualização de uma chamada pytest.raises no teste da CLI; e atualização de outra chamada pytest.raises no teste de JSON tags. Os testes direcionados passaram: 72 passed.
- Na nova análise, após o commit 3430fb3d e a atualização para a versão 1.1, o Quality Gate informou 0 novas issues e 0,0% de duplicação nas 4 linhas novas. A cobertura não recebeu um valor numérico porque nenhuma dessas linhas precisava ser coberta por testes.

Métrica	Antes	Depois	Variação
Violações	63	60	-3
Bugs	18	16	-2
Code smells	42	41	-1
Vulnerabilidades	3	3	0
New violations	—	0	Gate OK
New duplicated lines density	—	0,0%	Gate OK

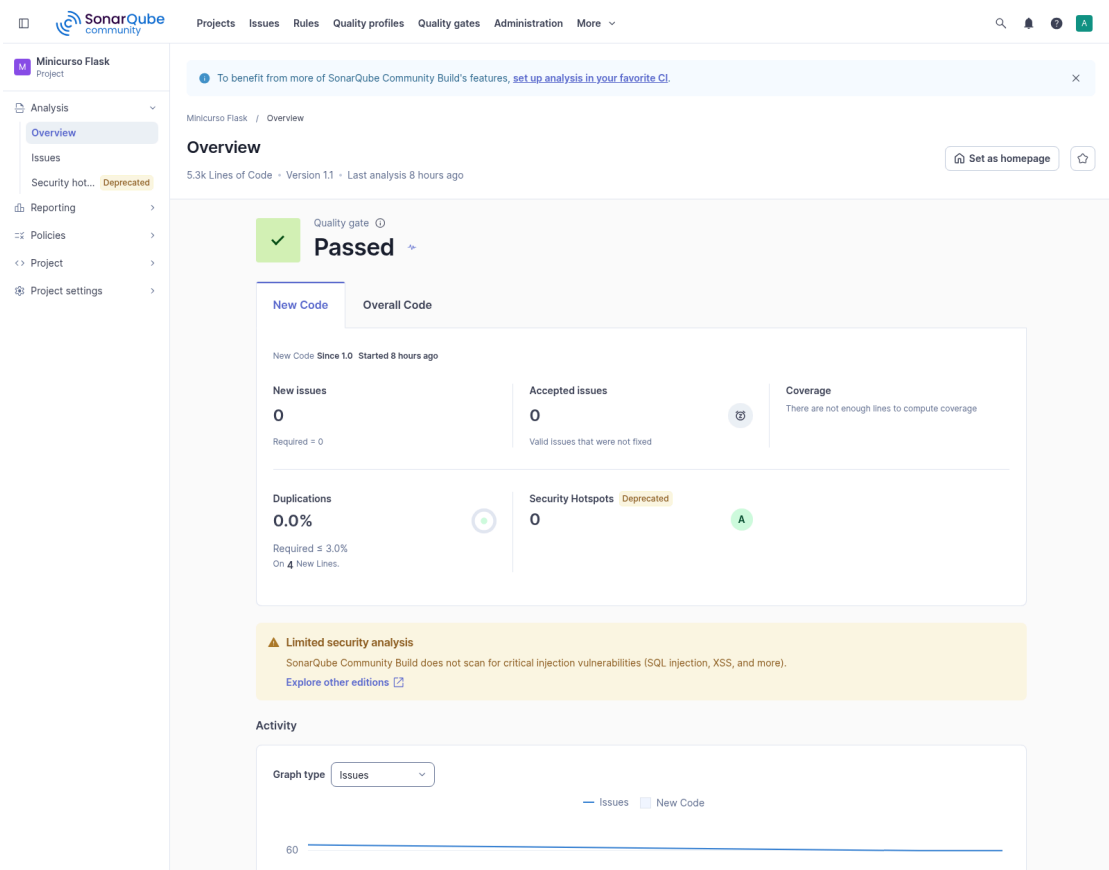


Figura 2 — Visão New Code após as três correções e a segunda análise.

1. Quadros quantitativos da visão geral (Overall) final

Métrica	Resultado final
Quality Gate	Passed / OK
Security	3 issues — D
Reliability	16 issues — E
Maintainability	52 issues — A
Accepted Issues	0
Coverage	0,0% — 3.246 linhas a cobrir
Duplications	0,3% — 28 linhas duplicadas
Security Hotspots	0 — A
Linhas de código	5.296 ncloc
Violações totais	60

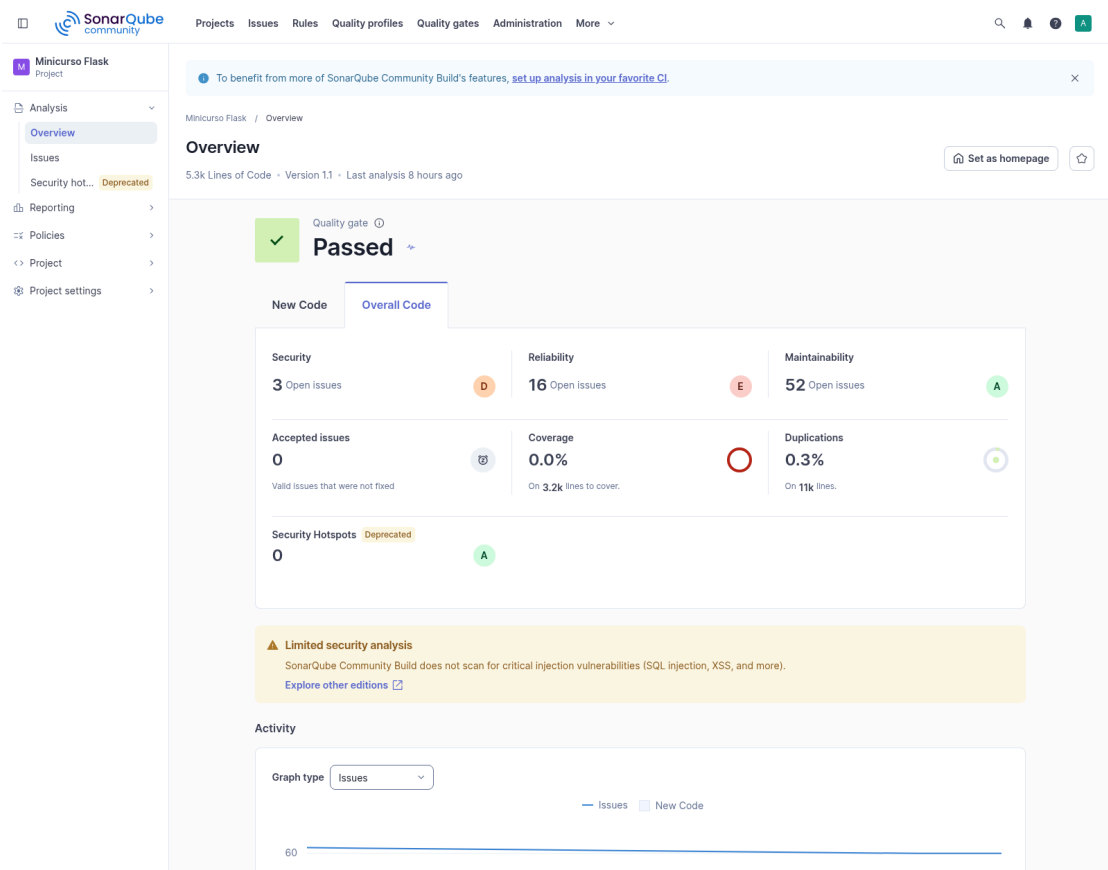


Figura 3 — Visão Overall final, após as correções.

- 1. A distribuição abaixo usa as severidades de impacto do modo MQR do SonarQube, exatamente nos níveis pedidos pelo template.

Severidade	Inicial	Final
Blocker	3	3
High	17	17
Medium	36	33
Low	7	7
Info	0	0
Total	63	60

- 1. Distribuição por categoria de Clean Code. As três correções reduziram a categoria Intentionality de 44 para 41:

Categoria	Inicial	Final
Consistency	4	4
Intentionality	44	41
Adaptability	14	14
Responsability (Responsibility)	1	1
Total	63	60

1. Estratégia de Redução de Issues Identificadas

A equipe tem cinco desenvolvedores em dedicação integral, com 8 horas por dia, 5 dias por semana e sprint de três semanas. A capacidade total é de 600 horas por sprint; o limite de 20% reserva até 120 horas para qualidade, equivalentes a 40 horas por semana. A ordem de trabalho será segurança, confiabilidade, manutenção, cobertura e duplicação.

- Maintainability Blocker/High = 0 — ainda não atingida; existem 15 impactos High de manutenção e nenhum Blocker.
- Security Blocker = 0 — atingida; as 3 issues de segurança são 1 High e 2 Low, sem Blocker.
- Security Hotspots = 0 — atingida; manter a revisão nas próximas alterações.
- Duplicidade de linhas = 0% — ainda não atingida; reduzir os 0,3% atuais.
- Cobertura de código $\geq 80\%$ — ainda não atingida; a cobertura atual é 0,0%.

Projeção: no Sprint 1 (120 h), tratar os riscos de segurança e confiabilidade, os Blockers e preparar um relatório de cobertura válido. No Sprint 2 (120 h), reduzir os impactos High de manutenção, eliminar duplicações e cobrir os módulos críticos. No Sprint 3 (120 h), ampliar a suíte para pelo menos 2.597 das 3.246 linhas cobráveis, executar a regressão completa e validar o Quality Gate. A estimativa é de até três sprints, ou nove semanas. A análise final ficou com Quality Gate aprovado para o código novo, 60 issues, 16 de confiabilidade, 3 de segurança, 52 de manutenção, 0 hotspots, 0,3% de duplicação e 0,0% de cobertura.

Relatório de Verificação

Análise Estática de Código

Nome: João Pedro Carvalho dos Santos

Turma: Desenvolvimento de Software Multiplataforma - 6º semestre Data:
12/09/2026

Projeto 2 - Ign-spring-api - <https://github.com/BrunoSerpa/spring-rest-data-security>

1. Detalhamento dos Passos de Execução da Análise

1. Executei o build e os testes com o perfil test e server.port=0. A suíte terminou com 381 testes aprovados, sem falhas, erros ou testes ignorados. Como o JaCoCo 0.8.8 é incompatível com o Java 21 usado no ambiente, essa etapa foi executada com jacoco.skip=true e o scanner não importou cobertura. Depois executei o scanner Maven na pasta do pom.xml, usando a chave br.edu.fatec:sprint-boot-sample. O token foi lido de uma variável de ambiente e não foi gravado no código nem neste relatório.

2. Visão Sumarizada (Overall)

1. Na análise inicial, antes das correções locais, o SonarQube registrou duas issues de manutenção: uma Low e outra Info.

Métrica	Resultado inicial
Quality Gate	Passed
Security	A
Reliability	A
Maintainability	A
Accepted Issues	0
Coverage	0,0% — 483 linhas a cobrir
Duplications	0,0%
Security Hotspots	0
Linhas de código	2.175 ncloc
Issues	2

- 1. A Figura 1 registra a visão Overall da análise inicial.

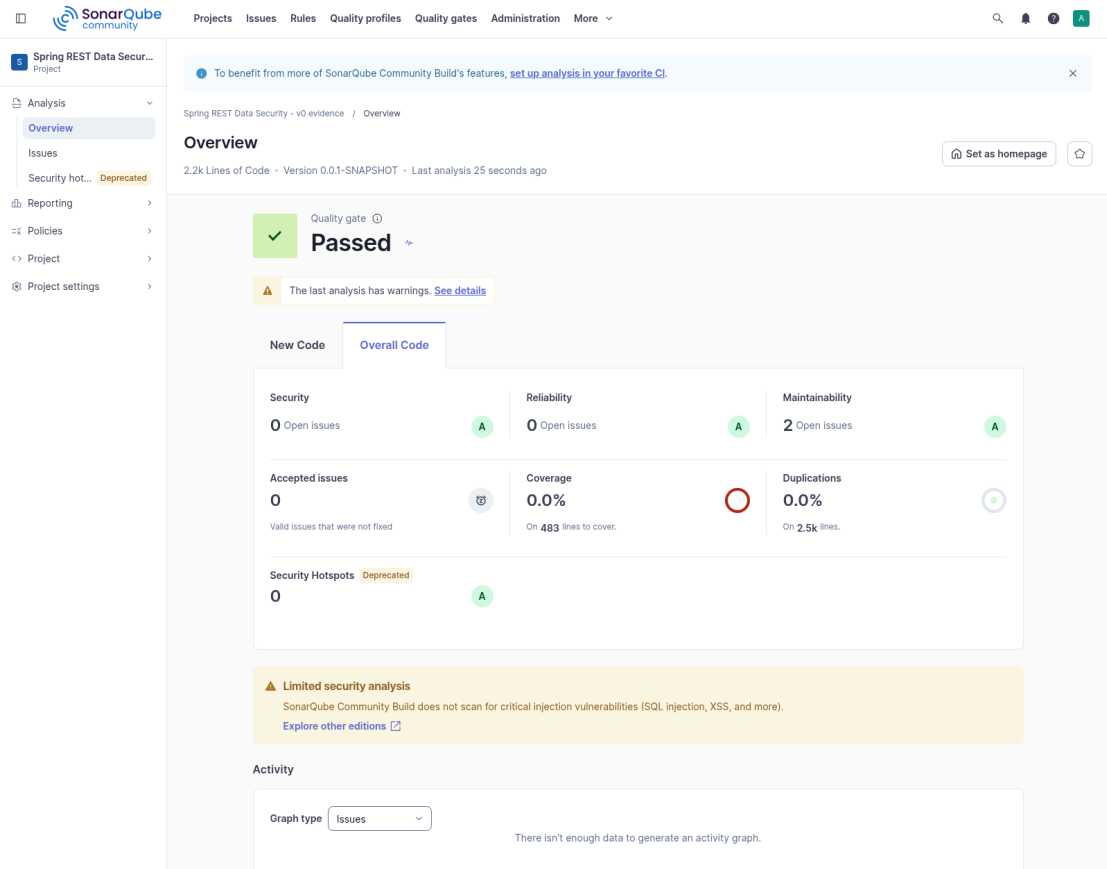


Figura 1 — Visão Overall inicial, antes das correções.

- 1. Quantidade total de linhas de código na análise inicial: 2.175 ncloc.

1. Visão Sumarizada (New Code Lines)

1. Foram feitas duas alterações pontuais. Em SecurityConfigTests.java, Mockito.mock foi substituído por um static import, corrigindo a issue S8924. Em JwtService.java, os cálculos passaram a usar Instant; Date permaneceu apenas na conversão exigida pela API do JJWT, portanto a issue S2143 continuou aberta. Os 381 testes permaneceram aprovados e uma nova análise foi processada pelo Sonar Scanner.
2. A análise final manteve o Quality Gate aprovado. O New Code ficou com 0 novas issues, 0 issues aceitas, cobertura de 0,0% em 5 linhas novas a cobrir, duplicação de 0,0% em 11 linhas novas e 0 Security Hotspots.

Métrica	Antes	Depois	Variação
Issues	2	1	-1
Code smells	2	1	-1
Bugs	0	0	0
Vulnerabilidades	0	0	0
Linhas de código	2.175	2.177	+2
Quality Gate	Passed	Passed	Mantido

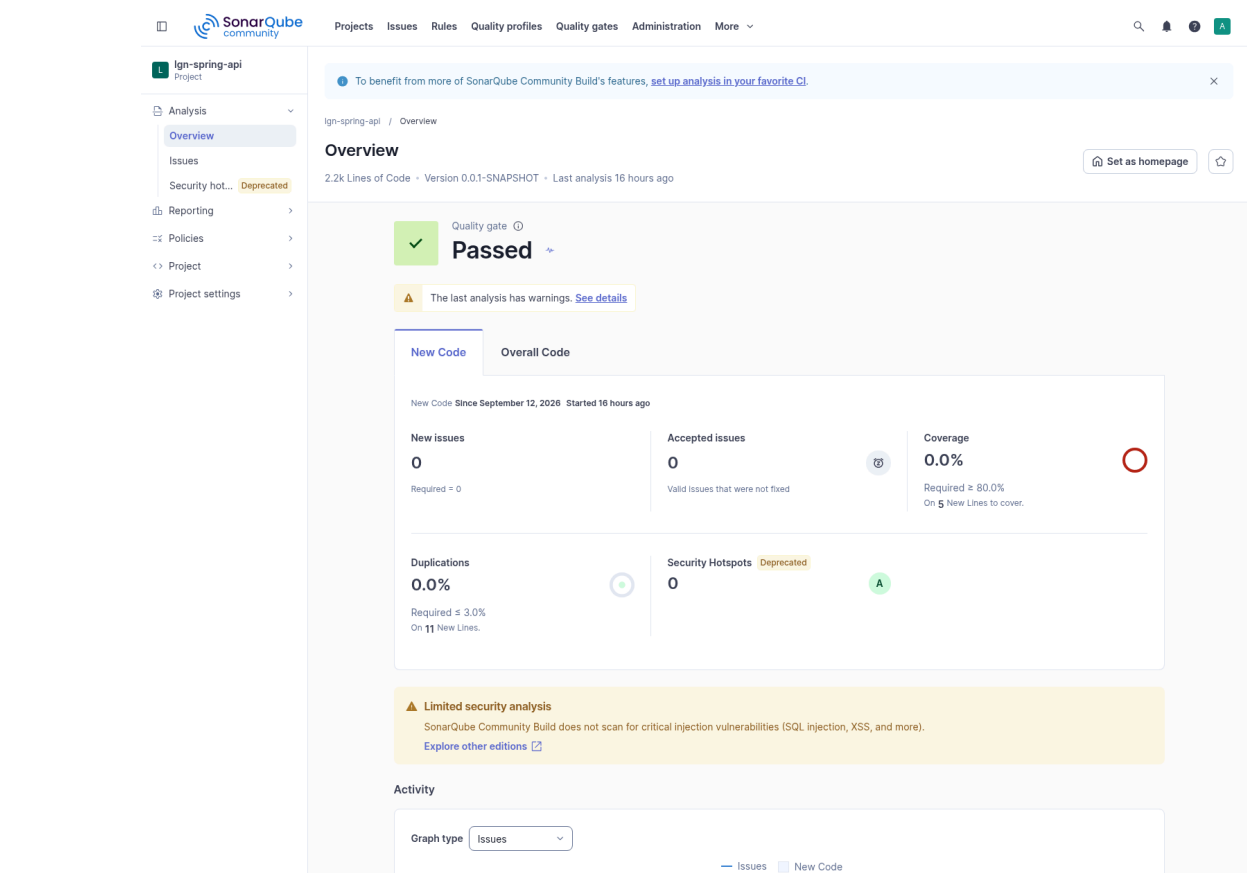


Figura 2 — Visão New Code após as correções e a segunda análise.

1. Quadros quantitativos da visão geral (Overall) final

Métrica	Resultado final
Quality Gate	Passed / OK
Security	A
Reliability	A
Maintainability	A
Accepted Issues	0
Coverage	0,0% — 484 linhas a cobrir
Duplications	0,0%
Security Hotspots	0
Linhas de código	2.177 ncloc
Issues	1

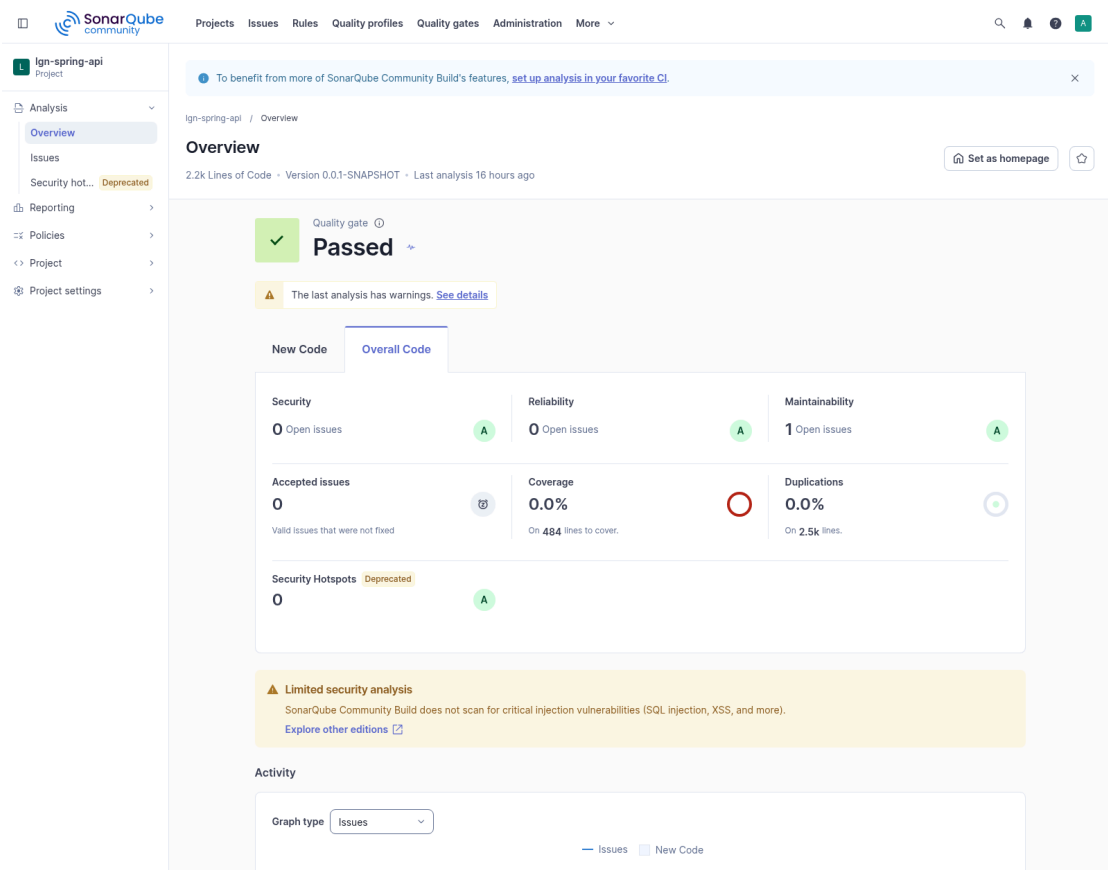


Figura 3 — Visão Overall final, após as correções.

- 1. Distribuição das issues por severidade de impacto:

Severidade	Inicial	Final
Blocker	0	0
High	0	0
Medium	0	0
Low	1	0
Info	1	1
Total	2	1

- 1. Distribuição por categoria de Clean Code Attribute:

Categoria	Inicial	Final
Consistency	1	1
Intentionality	1	0
Adaptability	0	0
Responsability (Responsibility)	0	0
Total	2	1

1. Estratégia de Redução de Issues Identificadas

Cinco desenvolvedores full-time têm capacidade conjunta de 600 horas em um sprint de três semanas. O limite de 20% permite dedicar até 120 horas por sprint, ou 40 horas por semana, às atividades de qualidade.

- Maintainability Blocker/High = 0 — atingida; manter o padrão no New Code.
- Security Blocker = 0 — atingida; revisar alterações relacionadas à segurança.
- Security Hotspots = 0 — atingida; verificar cada nova análise.
- Duplicidade de linhas = 0% — atingida; evitar duplicações nos próximos lotes.
- Cobertura de código $\geq 80\%$ — ainda não atingida; corrigir a geração do relatório JaCoCo e ampliar os testes.

Semana	Limite	Atividade
1	40 h	Corrigir o relatório JaCoCo e estabelecer a linha de base de cobertura.
2	40 h	Cobrir autenticação, filtro JWT e regras de segurança.
3	40 h	Cobrir recursos REST e persistência; executar a análise do primeiro sprint.
4	40 h	Ampliar os testes das classes abaixo da meta.
5	40 h	Executar regressão e fechar as lacunas restantes.
6	40 h	Executar a análise final e atualizar a documentação.

A projeção inicial é de seis semanas, divididas em duas sprints. Cada sprint usa no máximo 120 horas para qualidade. A estimativa deve ser revista ao final da terceira semana, quando o primeiro relatório JaCoCo válido mostrar quanto falta para cobrir pelo menos 80% das 484 linhas cobráveis.